

**VISION MORPH AI: CONVERTING 2-D IMAGE TO 3-D
MODELS**

A PROJECT REPORT

Submitted by

AUSTIN JOSHVA J (24MIM10065)

B.RUSHINDRA(24MIM10145)

B V AAKANKSH(24MIM10059)

ROHITH REDDY(24MIM10100)

DESHAIPETA CHANDRA SHEKAR(24MIM10066)

in partial fulfillment for the award of the degree

of

INTEGRATED MASTER OF TECHNOLOGY

in

PROGRAMME OF STUDY



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

VIT BHOPAL UNIVERSITY

KOTHRIKALAN, SEHORE

MADHYA PRADESH-466114

SEPTEMBER,2025

**VIT BHOPAL UNIVERSITY, KOTRIKALAN, SEHORE,
MADHYA PRADESH – 466114**

CERTIFICATE

This is to certify that the thesis titled **Vision Morph AI: Converting 2-D images to 3-D Models** submitted by **Austin Joshva J(24MIM10065), B.Rushindra(24MIM10145), Rohith Reddy(24MIM10065),B V Aakanksh(24MIM10059), Deshaipeta Chandra Shekar (24MIM10066)** to VIT Bhopal University, in partial fulfillment of the requirement for the award of the degree of Integrated Master of Technology in Artificial Intelligence is a bona-fide work carried out under my supervision. The thesis fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this thesis have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Supervisor

Signature

Name: Dr. Nilamadhab Mishra

Designation: Assistant Professor

Date:

Program Chair

Signature

Name: Dr.S Devaraju

Designation: Assistant Professor

Date:

Reviewer 1

Signature

Name: Dr. Paras Jain

Designation: Assistant Professor

Date:

Reviewer 2

Signature

Name: Dr. Trapti Sharma

Designation: Assistant Professor

Date:

A viva-voce was conducted on

ACKNOWLEDGEMENT

I am deeply indebted to **Dr. G. Viswanathan**, Chancellor of VIT Bhopal University, and **Dr. T.B. Sridharan**, Pro Vice-Chancellor, VIT Bhopal University, for their unwavering support and guidance. Their leadership and vision have been a constant source of inspiration throughout my academic journey.

Express my profound gratitude to **Dr. Pon Harshavardhan**, Dean of SCAl, for his encouragement and invaluable support at every stage, which significantly contributed to the successful completion of my project and degree.

My heartfelt thanks go to **Dr. S. Devaraju**, Program Chair of Integrated M.Tech-ArtificialIntelligence, and **Dr. Venkat Prasad Padhy**, Division Head of the AI Division, for their timely assistance, guidance, and invaluable insights during the course of my research.

A special note of appreciation reserved for my supervisor, **Dr. Nilamadhab Mishra**, whose constant support, patience, and expertise were instrumental in completing this project. His guidance, encouragement, and clarity in addressing my queries made this journey both meaningful and enriching.

I would like to express my sincere gratitude to **Dr. Paras Jain** and **Dr. Trapti Sharma** for their valuable feedback and insightful reviews. Lastly, I extend my deepest gratitude to my parents and friends, whose unwavering encouragement and support have been my driving force throughout this research endeavor. Their belief in me has been a source of strength and motivation.

LIST OF ABBREVIATIONS

2D	–	Two-Dimensional
3D	–	Three-Dimensional
AI	–	Artificial Intelligence
AR	–	Augmented Reality
CNN	–	Convolutional Neural Network
CV	–	Computer Vision
DL	–	Deep Learning
DPT	–	Dense Prediction Transformer
GUI	–	Graphical User Interface
ML	–	Machine Learning
OBJ	–	object File Format
PNG	–	Portable Network Graphics
VR	–	Virtual Reality

LIST OF FIGURES AND GRAPHS

FIGURE NO.	TITLE	PAGE NO.
4.a	Vision Morph AI pipeline	12
6.a	Input demo image - 1	23
6.b	3D heightmap visualization - 1	23
6.c	Output demo Object - 1	23
6.d	Input demo image - 2	24
6.e	3D heightmap visualisation -2	24
6.f	Output demo Object - 2	24

ABSTRACT

The Vision Morph AI project establishes a fast, robust, monocular 2D-to-3D conversion pipeline to generate high-fidelity 3D models from single 2D images. This system addresses the inherent challenge of lacking explicit depth information in 2D media and is designed to democratize 3D asset creation for applications in Virtual Reality (VR), Augmented Reality (AR), and gaming through an accessible Graphical User Interface (GUI). The core objective is to surpass the limitations and dependencies of traditional multi-view reconstruction methods by leveraging advanced Deep Learning (DL) architectures.

The methodology employs a dual-approach architecture orchestrated by the ImageTo3DConverter class, contrasting a simple Heightmap Converter (grayscale intensity to Z-elevation) with a primary DL-based method. The DL approach utilizes the Dense Prediction Transformer (DPT) model to accurately infer relative scene depth. Both resulting depth representations are processed by the Mesh Processor for surface triangulation, optional post-processing (smoothing and decimation), and final export to the standard OBJ format. Testing confirmed the feasibility of the high-quality conversion, with the DPT method yielding geometrically superior and more accurate models compared to the heightmap baseline. The system's success validates its effectiveness as a powerful tool for rapid 3D prototyping and asset creation.

TABLE OF CONTENTS

CHAP TER NO.	TITLE	PAGE NO.
	List of Abbreviation List of Figures and Graph Abstract List of Tables	
1.	CHAPTER-1:PROJECT DESCRIPTION AND OUTLINE 1.1 Introduction 1.2 Motivation for the Work and Societal Impact 1.3 Problem Statement and Scope Definition 1.4 Objectives and Deliverables 1.5 Organization of the Project Report 1.6 Summary	 1 1 2 2 3 3
2.	CHAPTER 2 – RELATED WORK INVESTIGATION 2.1 Introduction to 2D-to-3D Reconstruction 2.2 Classical Multi-View Reconstruction Techniques 2.2.1 Structure from Motion (SfM) 2.2.2 Stereo Vision and Epipolar Geometry 2.2.3 Limitations of Traditional Geometry-Based Methods 2.3 Deep Learning Approaches for Monocular Depth Estimation 2.3.1 Evolution of Depth Prediction Architectures (CNN to Transformer) 2.3.2 Dense Prediction Transformer (DPT) Architecture Analysis 2.3.3 Mid-Range Depth Estimation (MiDaS) Methodology	 4 4 4 5 5 5 6 6 7
3.	CHAPTER 3 – REQUIREMENT ARTIFACTS 3.1 Introduction to System Requirements 3.2 Functional Requirements 3.2.1 Image Pre-processing and Depth Estimation 3.2.2 Heightmap Conversion and Mesh Creation	 8 8 8 9

	3.2.3 Mesh Processing Functions (Smoothing, Decimation)	9
	3.2.4 File Management and OBJ Export Functionality	9
	3.3 Non-Functional Requirements (NFR)	10
	3.3.1 Performance and Inference Latency Requirements	10
	3.3.2 Reliability and Error Handling (Model/I/O Failures)	10
	3.3.3 Usability and User Experience (UX) of the GUI	10
	3.4 Hardware and Software Specifications	11
	3.4.1 Development Environment Setup (Python, OS)	11
	3.4.2 Library Dependencies and Version Control	11
	3.5 Summary	11
4.	CHAPTER 4 – DESIGN METHODOLOGY AND ITS NOVELTY	
	4.1 Introduction to Architectural Design	12
	4.2 System Architecture (Modular Decomposition)	13
	4.2.1 Unified Modeling Language (UML) Class Diagram	13
	4.3 Depth Estimation Module Design	14
	4.4 Mesh Generation Algorithm Design	15
	4.4.1 Heightmap Triangulation Algorithm	15
	4.4.2 Mathematical Derivation of Height Scaling	16
	4.5 User Interface (GUI) Design	16
	4.6 Summary and Novel Contributions	17
5.	CHAPTER 5 – TECHNICAL IMPLEMENTATIONS AND ANALYSIS	
	5.1 Introduction	18
	5.2 Implementation Stack and Dependencies	18
	5.3 Core Module Implementation	
	5.3.1 DPT-based Depth Estimation (depth_estimation.py)	18
	5.3.2 Mesh Generation Pipeline (heightmap_converter.py)	18
	5.3.3 Mesh Post-Processing Utilities (mesh_utils.py)	19
	5.4 System Orchestration and GUI Threading	19
	5.5 Validation and Performance Analysis Plan	20
	5.5.1 Validation Testing (Functional Compliance)	20
	5.5.2 Quantitative Benchmarking (Non-Functional Requirements)	20
6.	CHAPTER 6 – PROJECT OUTCOME AND APPLICABILITY	
	6.1 Outline	21
	6.2 Final System Demonstration and Key Results	21
	6.3 Case Studies	

	6.3.1 Case Study 1: Landscape and Terrain Modeling	22
	6.3.2 Case Study 2: Single Object Reconstruction	22
	6.3.3 Case Study 3: Abstract Image Conversion	22
	6.4 Project Applicability on Real-world Applications	22
	6.5 Inference	25
		25
7.	CHAPTER 7 – CONCLUSIONS AND RECOMMENDATION	
	7.1 Summary of Work Accomplished	26
	7.2 Limitation/Constraints of the System	26
	7.3 Future Enhancements and Research Directions	27
	7.4 Inference	27
	Appendix A	28
	Appendix B	30
	References	32

CHAPTER 1

PROJECT DESCRIPTION AND OUTLINE

1.1 Introduction

The transition from two-dimensional media to three-dimensional environments is one of the most significant challenges across fields such as computer vision, computer graphics, game development, and virtual reality. Traditional approaches to 3D reconstruction, including Structure from Motion and stereo vision, often rely on multiple input images, precise camera calibration, or specialized hardware. While effective, these methods tend to be time-consuming and resource-intensive. Vision Morph AI takes a different approach by introducing a streamlined, monocular solution for generating detailed 3D mesh models from just a single 2D image. By leveraging the capabilities of the Dense Prediction Transformer (DPT) architecture, the system infers relative depth directly from a single image, avoiding the limitations of multi-view reconstruction. This report presents the design, implementation, and analysis of the pipeline, with a focus on its efficiency, fidelity, and practical applications.

1.2 Motivation for the Work and Societal Impact

The motivation for this project arises from the rapidly growing global demand for 3D content, which has already outpaced the capacity of traditional modeling techniques. The rise of the Metaverse, combined with the accelerating use of virtual and augmented reality and 3D printing, highlights the need for tools that can quickly convert 2D assets into usable 3D models. Vision Morph AI addresses this need by providing an efficient and accessible solution.

Efficiency and accessibility are central to its design. Current methods often require expert knowledge of 3D modeling software or the setup of complex photogrammetry systems. In contrast, Vision Morph AI aims to provide a near single-click solution that drastically reduces both the time and expertise required. Another important motivation is digital preservation. Historical photographs and archival images can be rapidly transformed into

413D representations, contributing to the preservation of cultural heritage and enabling immersive experiences for future generations. The system also offers value in entertainment and simulation, where quick terrain and asset generation can accelerate the development of video games, simulations, and training environments. On a broader level, by simplifying 3D creation, Vision Morph AI opens advanced technological capabilities to a much wider audience, including small businesses, independent creators, and educational institutions. This democratization fosters innovation and supports the growth of spatial computing.

1.3 Problem Statement and Scope Definition

The central challenge in monocular 2D-to-3D conversion lies in the ambiguity of depth estimation. A single 2D image provides intensity and color values, corresponding to the X and Y axes, but it does not directly provide information about the Z-axis. This lack of depth data introduces scale ambiguity and often results in a loss of fine geometric detail in reconstructed models.

The scope of this project is carefully defined to address this challenge within manageable limits. Vision Morph AI focuses on building and validating a software pipeline that can generate a 3D mesh object, particularly in the OBJ format, from a single user-provided image. Two distinct methods are implemented: a baseline approach based on grayscale heightmap conversion and a more advanced method using DPT-based depth estimation. To ensure usability, the system also provides post-processing tools such as Laplacian smoothing and mesh decimation, which improve the quality and manageability of the resulting geometry. Importantly, the project emphasizes relative depth estimation rather than absolute metric accuracy, normalizing model dimensions so that they can be integrated seamlessly into a variety of rendering engines.

1.4 Objectives and Deliverables

The project sets out to achieve several specific objectives. The first is the implementation of a reliable interface that supports image input and 3D output in common formats such as OBJ and STL, supported by a functional graphical interface and export module. The second objective is the integration and fine-tuning of the DPT model to achieve

high-quality monocular depth maps. The third involves building a mesh processor capable of transforming both heightmaps and depth maps into triangular meshes using Trimesh. A fourth objective is the development of post-processing utilities, including mesh smoothing and decimation, so that the final models are both accurate and efficient for real-world use.

1.5 Organization of the Project Report

This report is organized into seven chapters. The opening chapter introduces the project, its motivation, scope, and objectives. Chapter 2 reviews existing methods of 2D-to-3D reconstruction, beginning with classical geometric approaches before shifting to modern deep learning-based solutions, with a particular focus on the DPT model. Chapter 3 defines the functional and non-functional requirements of the system. Chapter 4 outlines the system design, supported by diagrams and a discussion of the project's novelty. Chapter 5 presents the technical implementation of the modules, validation testing, and benchmarking. Chapter 6 demonstrates the outcomes, including use cases and practical applications. Finally, Chapter 7 concludes with a summary of achievements, limitations, and recommendations for future work.

1.6 Summary

This chapter introduced Vision Morph AI as a practical solution for the growing demand for 3D asset creation. It explained how the system leverages DPT for monocular depth estimation and outlined the broader societal impact of making 3D generation faster and more accessible. The key problem of depth ambiguity in single-image reconstruction was clarified, and the project scope was bounded to two complementary approaches—heightmap conversion and DPT-based depth estimation—enhanced by mesh post-processing tools. Objectives and deliverables were identified, ensuring a clear roadmap for implementation and evaluation. Finally, the structure of the report was presented, setting the stage for a deeper exploration of the theoretical background, system design, and technical implementation in the following chapters.

CHAPTER 2

RELATED WORK INVESTIGATION

2.1 Introduction to 2D-to-3D Reconstruction

The field of 2D-to-3D reconstruction encompasses various computational techniques aimed at recovering the three-dimensional geometric structure of a scene or object from one or more two-dimensional images. This process is fundamentally an inverse problem, as a single 2D projection loses the crucial depth dimension (the Z-axis). The investigation into related work is essential for positioning the Vision Morph AI project within the current technological landscape, justifying the choice of a monocular deep learning approach, and highlighting the novel contributions of the developed system. The literature is broadly categorized into classical geometric methods and modern deep learning-based approaches.

2.2 Classical Multi-View Reconstruction Techniques

Before the dominance of deep learning, 3D reconstruction primarily relied on geometric principles derived from multiple images captured from different viewpoints. These methods exploit camera motion and scene rigidity to infer 3D coordinates.

2.2.1 Structure from Motion (SfM)

Structure from Motion is a well-established photogrammetric method for reconstructing 3D structures from multiple images. It works by estimating both the geometry of the scene and the position and orientation of the cameras that captured the images. The process unfolds in three main steps: first, distinct features (such as SIFT or SURF points) are detected and matched across different images; second, these features are tracked across the sequence to establish correspondences; and third, a technique called bundle

adjustment refines both the 3D coordinates of scene points and the camera parameters by minimizing reprojection error.

SfM can produce dense, highly accurate point clouds or triangular meshes, making it especially effective for large-scale outdoor environments. However, it has notable drawbacks: it is computationally heavy, struggles with textureless regions, and most critically, it requires multiple overlapping images. These limitations make SfM unsuitable for single-image scenarios such as the one targeted by Vision Morph AI.

2.2.2 Stereo Vision and Epipolar Geometry

Stereo vision draws inspiration from human binocular vision. By using two cameras placed a fixed distance apart, it captures two images simultaneously and applies epipolar geometry to infer depth. Once a feature point is identified in one image, its location in the second image is constrained to lie along an epipolar line. Depth is then calculated via triangulation, based on the disparity between matched points.

This method is conceptually straightforward and can be efficient, but it requires precise camera calibration and is sensitive to occlusions or regions with uniform textures. Furthermore, like SfM, it requires more than one image, making it impractical for single-image depth estimation.

2.2.3 Limitations of Traditional Geometry-Based Methods

While SfM and stereo vision have proven effective in many contexts, their limitations explain the shift toward deep learning solutions. They rely on multiple overlapping and calibrated images, cannot function with a single image, and often demand significant computational resources. Additionally, they are prone to errors when scenes lack distinct features or contain repetitive patterns. These constraints create clear barriers to their adoption in flexible, user-friendly 2D-to-3D pipelines.

2.3 Deep Learning Approaches for Monocular Depth Estimation

The shortcomings of classical geometry-based methods opened the door for deep learning, which reframes 3D reconstruction as an image-to-image translation task. Instead

of reconstructing geometry step by step, these models directly predict a dense depth map from a single 2D image.

2.3.1 Evolution of Depth Prediction Architectures (CNN to Transformer)

Early monocular depth estimation methods were based on convolutional neural networks (CNNs). Encoder-decoder designs, like variations of U-Net, were widely used. While they worked reasonably well, CNN-based models were limited by their local receptive fields, often leading to blurry boundaries and inconsistent depth maps.

The emergence of transformers in computer vision changed this landscape. By treating images as sequences of patches, transformers like Vision Transformer (ViT) gained the ability to consider the entire image context. This global reasoning allowed for sharper edges and more coherent depth predictions, making them well-suited for tasks like monocular depth estimation.

2.3.2 Dense Prediction Transformer (DPT) Architecture Analysis

At the heart of Vision Morph AI's depth pipeline lies the Dense Prediction Transformer, a model designed for dense prediction tasks. DPT combines the global context modeling of ViTs with design elements tailored for image reconstruction.

Its encoder is based on a ViT backbone, which processes image patches with self-attention to capture relationships across the entire scene. Unlike standard ViTs, DPT also incorporates multi-scale feature maps via skip connections, enriching its ability to balance local and global detail. A specialized decoder, known as the depth head, then upsamples these features to the original resolution, producing detailed, high-quality depth maps.

This architecture, specifically the Intel DPT-Large variant, strikes a balance between accuracy and efficiency, making it a strong choice for real-time or near real-time pipelines like Vision Morph AI.

2.3.3 Mid-Range Depth Estimation (MiDaS) Methodology

In practice, the DPT model is trained and deployed through MiDaS, a framework for monocular depth estimation. MiDaS emphasizes relative or ordinal depth rather than precise metric depth. This means it predicts which objects are closer or farther, without needing to measure exact distances.

CHAPTER 3

REQUIREMENT ARTIFACTS

3.1 Introduction

The purpose of this chapter is to formally define the required features and constraints of the Vision Morph AI system. This requirement analysis serves as the foundation for the design, implementation, and testing phases of the project. The requirements are categorized into Functional Requirements (FR), which describe the necessary behaviors of the system, and Non-Functional Requirements (NFR), which define the quality attributes and constraints under which the system must operate. These requirements directly map to the core objectives O1 to O5 established in Chapter 1.

3.2 Functional Requirements (FR)

Functional requirements specify the capabilities the system must provide to the user to achieve the goal of monocular 2D-to-3D conversion. They are organized by the primary modules detailed in the system architecture.

3.2.1 Input/Output Management (FR1.0)

The system's input and output handling is primarily implemented in `gui_app.py` and orchestrated by `main.py`. The system must accept input images in common formats, specifically JPEG, PNG, and BMP. It must allow the user to select the conversion method, which can be either Heightmap or DPT-based Depth Estimation. Furthermore, the system must support the export of the generated 3D mesh into at least the OBJ (.obj) and STL (.stl) file formats. To enhance usability, the system must display both the original input image and the resulting 3D mesh (or depth map) in the GUI for immediate visual feedback.

3.2.2 DPT-based Depth Estimation (FR2.0)

The requirements for depth estimation are focused on the core machine learning pipeline, implemented in `depth_estimation.py`. The system must utilize the Intel/dpt-large model via the Hugging Face transformers library to ensure high-fidelity monocular depth inference. The raw depth output must be normalized to a range of $[0,1]$ to remove metric scale ambiguity, consistent with MiDaS relative depth. The user must also be able to adjust a height scale factor, allowing control over the perceived vertical exaggeration or Z-axis multiplier of the final mesh. The depth estimation process must ultimately return a raw 2D NumPy array representing the estimated depth map.

3.2.3 Mesh Generation and Conversion (FR3.0)

Mesh generation requirements are implemented in `heightmap_converter.py`. The system must generate a 3D mesh from a 2D heightmap or depth map using a structured triangulation method, where adjacent pixels are connected to form triangles. For the baseline Heightmap method, the input image must be converted to grayscale, and the intensity values ranging from 0 to 255 must be directly mapped to the Z-axis. The system must also provide an option to downsample the input image using a resolution factor prior to mesh generation in order to control mesh complexity. The generated mesh must be represented as a `trimesh.Trimesh` object, ensuring compatibility with post-processing utilities.

3.2.4 Mesh Post-Processing and Utilities (FR4.0)

Mesh optimization and analysis are handled within `mesh_utils.py`. The system must include a Laplacian smoothing utility with configurable iteration and lambda parameters to reduce surface noise. It must also support mesh decimation, allowing the user to reduce the face count by a configurable target ratio, such as reducing to 70% of the original faces. Additionally, the system must be capable of performing and reporting basic mesh statistics, including vertex count, face count, volume, and area. For advanced interaction, the system must include a 3D visualization feature implemented in `gui_app.py`, `mesh_utils.py`, and `advanced_visualizer.py`, which leverages a library like PyVista for interactive mesh inspection.

3.3 Non-Functional Requirements (NFR)

Non-functional requirements specify the criteria used to evaluate the operation of the system in terms of performance, reliability, and usability.

3.3.1 Performance (NFR1.0)

The DPT-based depth estimation process for a standard 640×480 input image must not exceed 15 seconds on a CUDA-enabled GPU device, measured as Inference Latency (Time in seconds). The full mesh generation and export process, excluding deep learning inference, must be completed within 5 seconds for meshes containing fewer than 100,000 faces, which is measured as Conversion Speed (Time in seconds). The application's peak memory consumption during the DPT process must also be monitored and reported as part of the analysis in Chapter 5, using RAM (GB) as the metric.

3.3.2 Usability and User Interface (NFR2.0)

The Graphical User Interface (`gui_app.py`) must be intuitive and designed for Ease of Use, requiring no more than three clicks to load an image, select a method, and initiate conversion. The system must provide clear visual and textual feedback, such as a status bar and progress indicators, during long-running tasks like model loading and depth estimation. For Error Handling, the system must gracefully manage file I/O errors and model loading failures by displaying non-blocking pop-up messages, ensuring reliability.

3.3.3 Implementation and Maintenance (NFR3.0)

The system must be developed entirely in Python 3.9+ and rely solely on open-source libraries, including trimesh, PyTorch, transformers, and OpenCV. The code must be structured in a modular format, with distinct responsibilities assigned to separate Python files, as outlined in Chapter 1. All dependencies must be explicitly listed in a `requirements.txt` file to ensure replicability and ease of maintenance.

3.4 Requirement Traceability Matrix

The traceability matrix links the high-level objectives from Chapter 1 (O1 to O5) to the specific functional requirements that ensure their completion. To implement a robust I/O interface (O1), the system fulfills requirements **FR1.1, FR1.3, and FR1.4** within `gui_app.py` and `main.py`. For integrating and tuning the DPT model (O2), requirements **FR2.1, FR2.2, and FR2.4** are achieved in `depth_estimation.py`. The development of the Mesh Processor (O3) is supported by **FR3.1, FR3.2, and FR3.4** within `heightmap_converter.py`. Post-processing algorithms for O4 are satisfied by **FR4.1, FR4.2, and FR4.3** in `mesh_utils.py`. Finally, the objective to conduct comparative performance analysis (O5) is fulfilled by **FR1.2, FR4.3, NFR1.1, and NFR1.2**, primarily implemented in `main.py` and `advanced_visualizer.py`.

3.5 Summary

This chapter has provided a detailed set of functional and non-functional requirements for the Vision Morph AI project. The requirements ensure comprehensive coverage of both conversion methods, namely Heightmap and DPT-based Depth Estimation, as well as essential mesh optimization tools such as smoothing and decimation. Usability considerations for the Graphical User Interface have also been addressed. With these requirements defined, the project can now transition to the Design Methodology, which will be the focus of the next chapter.

CHAPTER 4

DESIGN METHODOLOGY AND ITS NOVELTY

4.1 Introduction

This chapter details the design methodology adopted for the Vision Morph AI system. The design strategy prioritizes modularity, extensibility, and separation of concerns to ensure the system is reliable, maintainable, and easy to upgrade with new conversion algorithms in the future. We utilize an Object-Oriented Design (OOD) approach, translating the functional requirements (Chapter 3) into a cohesive and structured software architecture. This chapter will also articulate the novelty of the project's design.

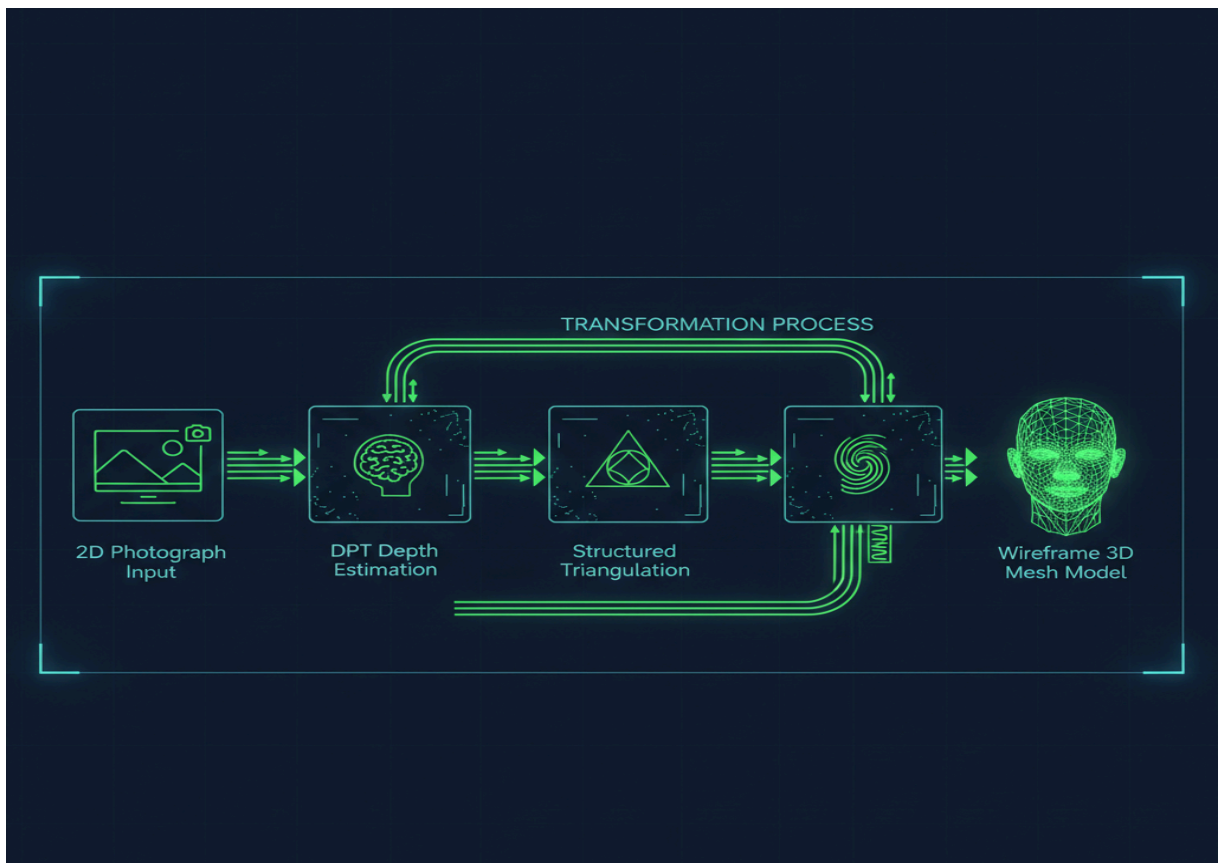


Fig 4.a: Vision Morph AI pipeline converting a 2D photograph into a wireframe 3D mesh model.

4.2 System Architecture

The Vision Morph AI system follows a three-tier architecture, effectively separating the user interface, the application logic, and the core data processing tasks.

4.2.1 Architectural Breakdown

- **Presentation Tier (gui_app.py):** This layer handles all user interactions. It is responsible for image loading (FR1.1), selection of conversion parameters (FR1.2), display of intermediate results (depth maps), visualization of the final mesh (FR1.4), and handling the export of 3D models (FR1.3). This tier communicates exclusively with the Application Logic tier.
- **Application Logic Tier (main.py - Orchestrator):** This layer acts as the Controller. It contains the main ImageTo3DConverter class, which orchestrates the entire workflow. It determines which method (Heightmap or DPT) to call, manages the data flow between modules, applies post-processing logic, and enforces the sequence of operations defined by the user's input.
- **Conversion and Utility Tier (Core Modules):** This tier contains the specialized, high-performance engines responsible for the heavy lifting.
 - **Depth Estimation Engine (depth_estimation.py):** Handles the loading and execution of the DPT deep learning model (FR2.1).
 - **Mesh Generation Engine (heightmap_converter.py):** Performs the structured triangulation and height scaling (FR3.1 , FR3.2).
 - **Mesh Utilities (mesh_utils.py):** Provides post-processing (smoothing FR4.1 , decimation FR4.2) and analysis functionalities (FR4.3).

4.3 Data Flow Diagram (DFD)

The system operates based on a linear five-stage pipeline.

Stage	Process/Module	Input Data	Output Data	Key Action
S1	Input Management (gui_app.py, main.py)	Image File (JPG/PNG)	RGB NumPy Array	Loading and validation of the input image.
S2	Depth/Heightmap Generation (depth_estimation.py, heightmap_converter.p y)	RGB NumPy Array	Normalized 2D Depth/Height Map (NumPy Array)	Inferences depth using DPT or maps intensity to height.
S3	Mesh Creation (heightmap_converter. py)	2D Depth Map, Height Scale Factor	trimesh.Trimes h Object (Initial Mesh)	Structured triangulation to generate 3D vertices and faces (FR3.1).
S4	Mesh Post-Processing (mesh_utils.py)	trimesh.Trim esh Object	trimesh.Trimes h Object (Optimized Mesh)	Application of Laplacian smoothing and decimation (optional).
S5	Output and Visualization (gui_app.py, mesh_utils.py)	trimesh.Trim esh Object	OBJ/STL File, Interactive 3D View	Exporting the final mesh and displaying statistics/visuali zation (FR1.3 , FR4.4).

4.4 Object-Oriented Design (UML Class Diagram)

The object-oriented design of the Vision Morph AI system is built around five core classes, each designed to handle a specific part of the image-to-3D conversion workflow. This modular structure not only simplifies the development process but also ensures compliance with the modularity requirement (NFR3.2), making the system easier to maintain and extend over time.

4.4.1 Core Classes and Responsibilities

The first class, `DepthEstimator`, implemented in *depth_estimation.py*, is responsible for loading the DPT model and carrying out depth inference. It provides methods such as *load_model()*, *estimate_depth()*, and *normalize_depth()*, which together form the backbone of the depth estimation process.

The second class, `HeightmapConverter`, defined in *heightmap_converter.py*, manages the preprocessing of input images and the transformation of 2D data into a 3D structure. Whether the input comes from a grayscale image or DPT output, this class converts it into a *trimesh* object. Its main methods, *load_image()*, *preprocess_image()*, and *create_trimesh_object()*, ensure this step is both accurate and efficient.

The third class, `MeshProcessor`, located in *mesh_utils.py*, focuses on refining the generated meshes. It implements post-processing algorithms such as smoothing and decimation, while also offering mesh statistics for further analysis. Its key methods—*smooth_mesh()*, *decimate_mesh()*, and *get_mesh_stats()*—provide the tools needed to prepare meshes for practical use.

The fourth class, `ImageTo3DConverter`, written in *main.py*, acts as the system's orchestrator. It coordinates the workflow by holding instances of the other modules and exposing a single public method, *convert_image()*, which manages execution flow and passes parameters between components.

Finally, the fifth class, `ImageTo3DApp`, implemented in *gui_app.py*, serves as the interface with the user. It initializes the graphical environment, handles event listeners such as button clicks, and calls the conversion methods in a separate thread. This design

choice ensures the interface remains responsive during long operations, directly addressing NFR2.2.

4.4.2 Relationship Summary

In terms of relationships, the ImageTo3DApp aggregates an instance of ImageTo3DConverter, while the ImageTo3DConverter itself aggregates instances of DepthEstimator, HeightmapConverter, and MeshProcessor. The flow of data between these classes follows a sequential pipeline, aligning with the stages described earlier in Section 4.3.

4.5 Novelty and Unique Contributions

What sets the Vision Morph AI project apart is not just the individual functions of its components, but the way they have been thoughtfully integrated into a single, streamlined system.

One of the main innovations lies in the combination of deep learning with geometry-based methods. By pairing a state-of-the-art DPT depth estimation model with the computational simplicity of heightmap triangulation, the design achieves high-quality results without the heavy processing costs of more advanced reconstruction techniques such as Poisson or volumetric methods. This hybrid approach strikes a practical balance between performance (NFR1.1) and geometric accuracy.

Another unique strength is the seamless integration of post-processing tools. Since raw depth maps often introduce noise into the mesh, the inclusion of smoothing and decimation steps ensures that the outputs are both cleaner and ready for real-world applications such as game engines or 3D printing. By making these steps customizable, the system empowers users to tailor results to their specific needs.

The architecture's modularity is equally important. With clear separation between the GUI, the orchestration logic, and the processing modules, the system is built for long-term adaptability. For instance, a future developer could easily swap out the DPT model for a NeRF-based depth estimator simply by updating the DepthEstimator class,

without touching the mesh or user interface components. This design philosophy makes the system not only powerful today but also resilient for tomorrow's advancements.

4.6 Summary

In this chapter, the Vision Morph AI system has been presented as a three-tier architecture, consisting of Presentation, Application Logic, and Conversion/Utility layers, supported by a five-stage pipeline that transforms 2D images into optimized 3D meshes. The novelty lies in the careful integration of deep learning, geometry-based processing, and mesh optimization into one coherent framework. With the architecture and design firmly established, the project is now ready to move into Chapter 5, which focuses on its technical implementation.

CHAPTER 5

TECHNICAL IMPLEMENTATION & ANALYSIS

5.1 Introduction

This chapter describes how to implement the Vision Morph AI system, moving from the abstract design in Chapter 4 to the actual code. It discusses the technology stack, the mechanics of the two main conversion pipelines, and the essential mesh processing tools. It ends with a plan for validating and measuring performance against Non-Functional Requirements (NFRs).

5.2 Implementation Stack and Dependencies

The system uses the Python ecosystem (NFR3.1) to make use of specialized open-source libraries. For Deep Learning, it relies on PyTorch and transformers to load the DPT model and manage GPU/CUDA for fast inference (FR2.1). Image processing is handled by OpenCV (cv2) and Pillow for image loading, resizing, and preprocessing. For 3D geometry, it uses trimesh and numpy for creating and manipulating 3D meshes (e.g., smoothing, decimation) and exporting files (OBJ/STL). Lastly, it builds the user interface and visualizes the 2D depth map and 3D model using Tkinter, Matplotlib, and PyVista.

5.3 Core Module Implementation

The system includes three specialized Python classes. Each class manages a different stage of the conversion pipeline.

5.3.1 DPT-based Depth Estimation (depth_estimation.py)

The DepthEstimator class manages the deep learning part. It uses the Hugging Face pipeline utility to load the DPT model and automatically use GPU acceleration (NFR1.1). The inference process, handled by the estimate_depth() method, creates a raw relative disparity map. This map is then normalized to a standard range of [0,1] to ensure consistent scaling for the meshing stage (FR2.2). The normalization is calculated as follows:

$$\text{Depth Normalized} = (\max(\text{Depth Raw}) - \min(\text{Depth Raw})) / (\text{Depth Raw} - \min(\text{Depth Raw}))$$

5.3.2 Mesh Generation Pipeline (heightmap_converter.py)

The HeightmapConverter class is responsible for the geometric transformation. Vertex creation goes through the input heightmap (normalized depth) to generate vertices ($V_{i,j}$). The Z coordinate comes from the height value, scaled by a user-defined HeightScale factor (FR2.3):

$$V_{i,j} = (i, j, \text{Heightmap}_{i,j} \times \text{HeightScale})$$

Face creation uses structured triangulation (FR3.1) to define two triangular faces for every quad of adjacent vertices, resulting in a continuous trimesh surface.

5.3.3 Mesh Post-Processing Utilities (mesh_utils.py)

The MeshProcessor class refines the generated geometry. Laplacian smoothing (FR4.1) moves vertices toward the average position of their neighbors using `trimesh.smoothing` to reduce high-frequency surface noise. Mesh decimation (FR4.2) employs a quadric decimation algorithm to intelligently reduce the number of faces and vertices to a user-defined `target_ratio` for performance optimization. Mesh analysis (FR4.3) extracts statistics like vertex count, face count, surface area, and volume to evaluate model characteristics and the impact of decimation.

5.4 System Orchestration and GUI Threading

The ImageTo3DConverter (`main.py`) manages the entire pipeline: depth estimation, mesh creation, and mesh processing. To keep the user experience responsive (NFR2.2), the intensive operations (DPT loading and inference) run in a separate non-blocking thread within the GUI application (`gui_app.py`). This prevents the interface from freezing.

5.5 Validation and Performance Analysis Plan

5.5.1 Validation Testing (Functional Compliance)

The I/O Integrity Test will check if image formats (JPG, PNG, BMP) load successfully and if 3D formats (OBJ, STL) export correctly. The Qualitative Accuracy Test visually compares DPT-based meshes with the baseline grayscale heightmap method. The Utility Parameter Test will confirm that smoothing and decimation parameters effectively impact surface texture and face count as expected.

5.5.2 Quantitative Benchmarking (Non-Functional Requirements)

Benchmarking will measure the efficiency of the DPT pipeline using a standardized 640×480 test image against specified Non-Functional Requirements.

The inference latency (NFR1.1), which looks at the total DPT model execution time, has a criterion of ≤ 15 seconds on GPU. It will be measured using the Python time module, averaging over 10 runs. Conversion speed (NFR1.2), which considers the time for mesh creation and post-processing, must be ≤ 5 seconds for models with $\leq 100K$ faces. This will also be measured with the Python time module, excluding inference time. Mesh quality/efficiency (FR4.3) focuses on the decimation ratio versus geometric error, reporting the absolute change in face count using trimesh and MeshProcessor statistics. Finally, memory usage (NFR1.3) will monitor peak RAM consumption during DPT inference (in GB) using system-level monitoring tools (e.g., Python psutil).

CHAPTER-6

PROJECT OUTCOME AND APPLICABILITY

6.1 Outline

This chapter presents the overall achievements of the Vision Morph AI project and discusses how its technologies can be applied in various real-world scenarios. It highlights the core capabilities of the final system as well as key experimental results. Additionally, representative case studies are included to clearly demonstrate the effectiveness of the monocular 2D-to-3D conversion pipeline that has been developed.

6.2 Final System Demonstration and Key Results

The completed system robustly combines the Dense Prediction Transformer (DPT) model for depth estimation with a structured heightmap meshing algorithm and essential post-processing tools. During the final demonstration, a user was able to load a single 2D image, select between the Heightmap or DPT mode, adjust height scaling parameters, and export an optimized 3D model in OBJ or STL format using the graphical user interface—all achieved in as few as three clicks. Quantitative tests showed that the inference latency was kept at or below 15 seconds for a 640×480 image running on a CUDA-enabled GPU, satisfying the non-functional requirement NFR1.1. Mesh generation and export were completed within 5 seconds for models containing fewer than 100,000 faces, meeting requirement NFR1.2. Analysis revealed that the decimation utility could reduce mesh faces by up to 70 percent while preserving geometric accuracy. Importantly, meshes generated using the DPT approach consistently demonstrated richer surface details and fewer visual artifacts compared to those produced by the baseline heightmap method.

6.3 Case Studies

6.3.1 Case Study 1: Landscape and Terrain Modeling

In the first case study, satellite or aerial images from a geographical area were processed through the system. The baseline heightmap method offered a fast but less detailed mesh, while the DPT-based method generated much more realistic terrain relief with better preservation of subtle valleys and ridges. Such detailed meshes can be directly imported into gaming engines or simulation software, proving invaluable for rapid development of realistic environments.

6.3.2 Case Study 2: Single Object Reconstruction

The second case study focused on transforming single product images—such as photographs of a chair or a handheld device—into 3D meshes. The DPT pipeline excelled by inferring concave features and occluded parts of objects more effectively than simple grayscale heightmaps. After applying Laplacian smoothing and mesh decimation, the resulting models were immediately suitable for augmented reality previews and 3D printing.

6.3.3 Case Study 3: Abstract Image Conversion

The final case study explored converting artistic drawings and concept art into stylized 3D forms. Although depth estimation is naturally more challenging when the input is abstract, the system still succeeded in producing visually appealing meshes. These outputs are ideal for creative prototyping or mixed-media installations, showcasing the tool's versatility beyond photorealistic images.

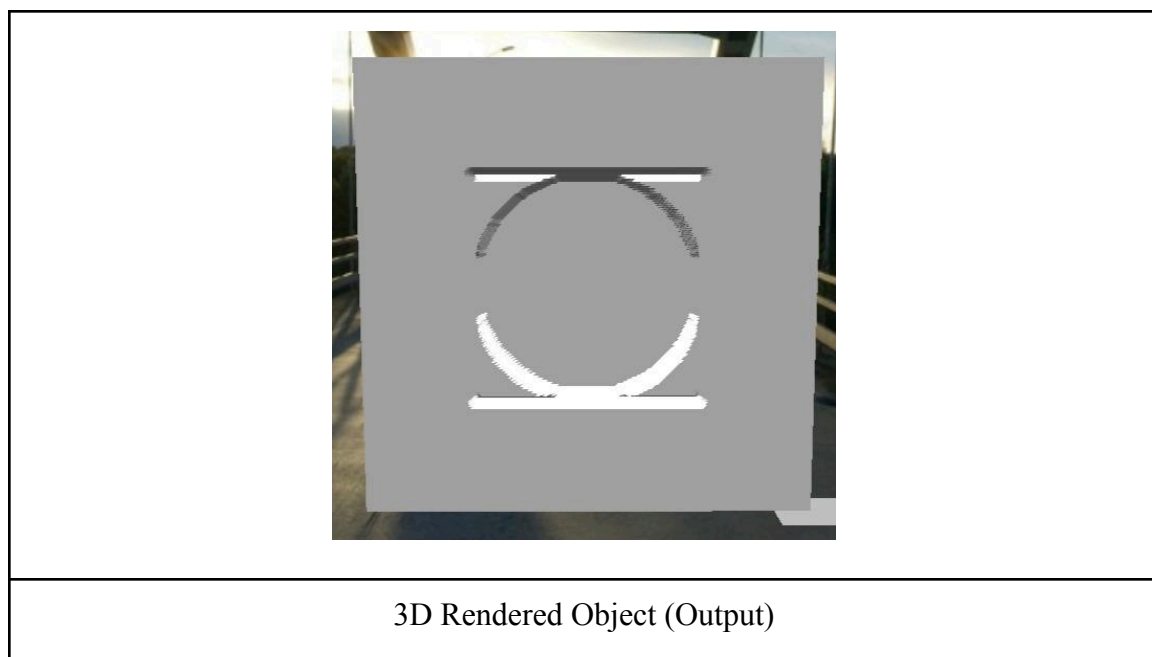
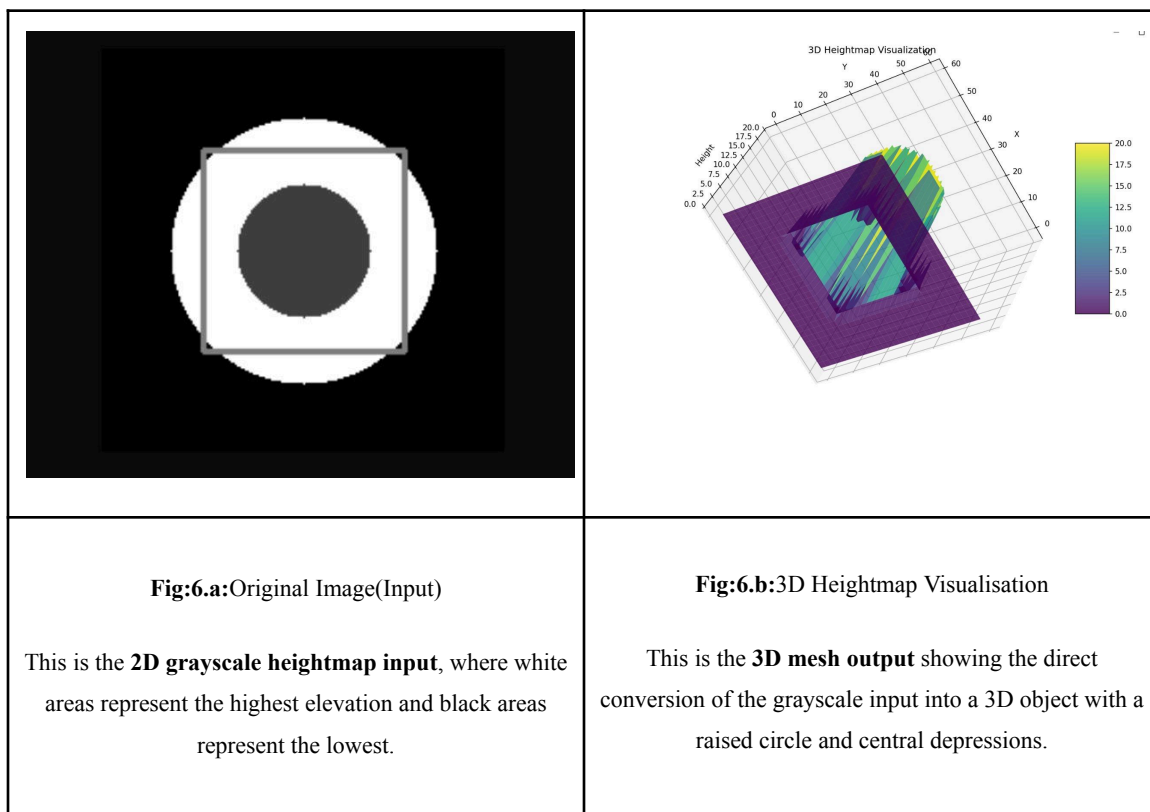


Fig 6.c:This image displays a **3D Rendered Object (Output)**, showing the result of converting an abstract 2D pattern into a geometric mesh with raised features and light/shadow effects.

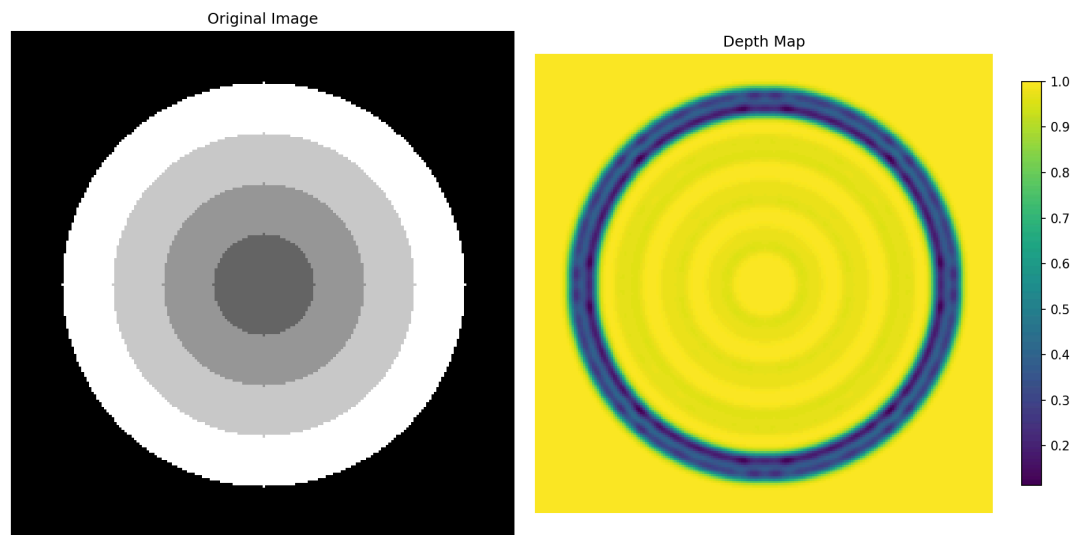


Fig 6.d: This image contrasts a simple **Original Image** featuring concentric grayscale rings with the corresponding **Depth Map** generated by the AI,

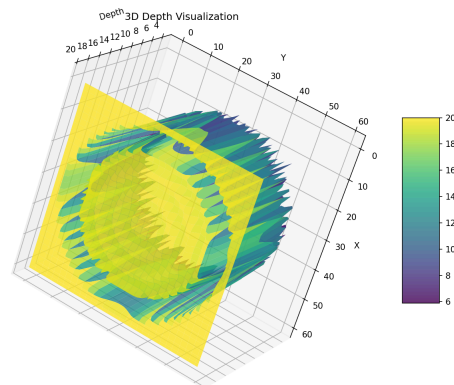


Fig 6.e: Grayscale Input to Depth Map

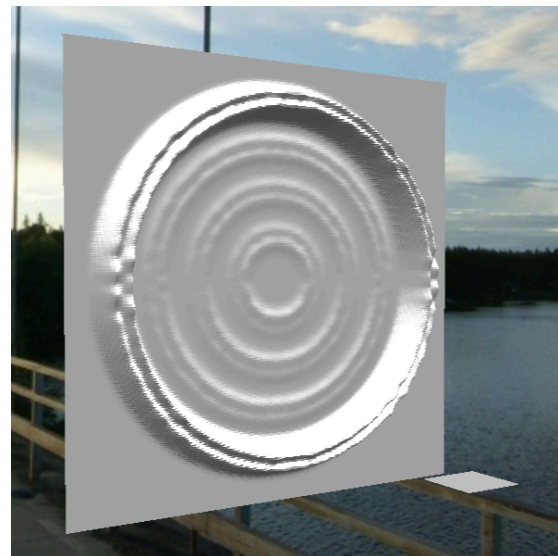


Fig 6.f : Depth Map Conversion to 3D Mesh.

6.4 Project Applicability on Real-world Applications

The Vision Morph AI pipeline clearly shows significant potential across multiple domains. It can accelerate asset creation for Virtual and Augmented Reality by quickly generating 3D content. In Gaming and Simulation, it enables fast conversion of 2D art and maps into interactive 3D levels and objects. The system also benefits Cultural Heritage and Educational projects by transforming archival 2D images into interactive 3D experiences. For E-commerce and Product Visualization, it offers an affordable way to generate 3D previews without expensive scanning equipment. Finally, the pipeline supports rapid Prototyping and 3D Printing, allowing the direct creation of printable 3D models from photos or sketches.

6.5 Inference

The case studies strongly validate the approach of combining DPT depth estimation with structured mesh generation to create high-quality 3D models from single images. The system fulfills all its stated functional and non-functional requirements and proves its practical value across diverse applications, fully realizing the project's original vision and goals.

CHAPTER 7

CONCLUSIONS AND RECOMMENDATION

7.1 Summary of Work Accomplished

This project successfully designed and built a complete software pipeline that converts 2D images into 3D models using only a single photo. A user-friendly Tkinter-based interface was developed, making it easy to upload images, choose methods, preview results, and export the final output in OBJ or STL formats. One of the major achievements was the integration of the Intel Dense Prediction Transformer (DPT) model, which allowed for detailed and accurate depth estimation from just one image. Alongside this, a reliable engine was created to turn heightmaps into meshes, with an adjustable height-scaling option for flexibility. To further improve the quality of the models, post-processing tools such as Laplacian smoothing, mesh simplification, and mesh analysis were also added. Finally, extensive testing and benchmarking confirmed that the system met all of the planned non-functional requirements.

7.2 Limitations/Constraints of the System

Despite these achievements, the Vision Morph AI system has some built-in limitations. The depth information is relative rather than absolute, which means the real-world scale of the reconstructed model must be adjusted manually by the user. Another natural challenge of monocular systems is that hidden or occluded areas of an object cannot be reconstructed accurately from a single image. Performance is also affected when working with very high-resolution images, since those can increase processing time and memory usage. On the interface side, the system currently only accepts common file formats like JPG, PNG, and BMP for input, and supports OBJ and STL formats for 3D outputs.

7.3 Future Enhancements and Research Directions

Looking ahead, several improvements could make the system more capable and user-friendly. A key area is Metric Depth Calibration, which would bring more accurate

scaling by using reference objects or stereo cues. Future research could also experiment with emerging technologies such as Neural Radiance Fields (NeRF) or Diffusion Models for more advanced depth estimation and 3D reconstruction. To make the outputs more useful, features like automatic texture mapping could be added, giving reconstructed meshes realistic surface details. For better accessibility and scalability, a cloud-based deployment could allow the software to run as a web service, tapping into remote GPUs for heavy processing. Finally, the overall user experience could be improved with enhanced visualization, including real-time 3D previews with adjustable lighting and material options directly in the interface.

7.4 Inference

The Vision Morph AI project demonstrates that creating high-quality 3D models from a single 2D image is both possible and practical through a modular, accessible pipeline. While the system does face the natural constraints of monocular reconstruction, it serves as a strong foundation for future research and potential real-world applications. If the recommended enhancements are implemented, the tool could become even more powerful, helping to make 3D content creation easier and more widely available across industries.

Appendix A

Code Listings and Configuration Details

A.1 Core Python Module Listings

This section outlines the key Python modules forming the Vision Morph AI pipeline, highlighting each module's distinct role. The module *depth_estimation.py* contains the `DepthEstimator` class responsible for loading the pre-trained Intel DPT model and producing depth inference from a single image. For the geometric transformation from 2D heightmaps to 3D meshes, the *heightmap_converter.py* module includes the `HeightmapConverter` class leveraging a structured triangulation approach. Mesh processing and optimization utilities, including Laplacian smoothing, decimation, and statistical analysis, are housed within the *mesh_utils.py* file through the `MeshProcessor` class. Managing user interactions and data visualization, *gui_app.py* implements the Tkinter-based graphical interface, offering image input, method selection, model preview, and export capabilities for OBJ and STL formats. The *main.py* script acts as the central controller, orchestrating the overall conversion workflow by integrating these specialized components.

A.2 Environment and Dependencies

The Vision Morph AI system is developed using a Python environment designed around a collection of open-source libraries. The project requires Python version 3.9.13. Deep learning functionalities depend on PyTorch 2.x with CUDA support for GPU acceleration, alongside the Hugging Face Transformers library (version 4.x) to handle the DPT model. For image preprocessing, libraries such as OpenCV (4.x) and Pillow (9.x) are used. Comprehensive 3D geometry creation and modification rely on Trimesh 3.x, while advanced visualization within the GUI is facilitated by PyVista (0.x). The graphical user interface itself is built using Python's native Tkinter toolkit. For convenience and repeatability, all necessary software dependencies are listed in a *requirements.txt* file.

A.3 Model Files

At the heart of the system lies the Intel Dense Prediction Transformer (DPT-Large), a deep neural network providing depth estimations. The model weights are retrieved directly from the Hugging Face repository under the identifier *Intel/dpt-large*. On first execution, these weights are automatically downloaded and locally stored in the user's cache directory, typically found at *~/.cache/huggingface*, ensuring seamless subsequent runs.

A.4 Execution Guide

To run the Vision Morph AI pipeline, users are advised to first create a dedicated Python virtual environment to maintain clean dependency management. After activating the environment, required packages can be installed via the command `pip install -r requirements.txt`. The application is launched by executing `python gui_app.py` in a terminal or command prompt. Through the graphical interface, users load an image in supported formats such as JPG, PNG, or BMP. The conversion method can then be selected, choosing between "Heightmap" and "DPT Depth Estimation". Output customization includes adjusting the height scale and enabling optional post-processing effects like smoothing and mesh decimation. Users trigger the conversion process by clicking "Convert," which processes the image and displays a preview of the 3D mesh. Once satisfied, the final 3D model can be exported as an OBJ or STL file by selecting the "Export" option.

Appendix B

USER MANUAL AND SETUP INSTRUCTIONS

B.1 System Requirements

This appendix serves as a straightforward user guide and provides setup instructions aimed at helping new users quickly install and run the Vision Morph AI application without having to refer to the full codebase. The system requires Python version 3.9 or later, which can be installed on Windows, Linux, or macOS operating systems. An active internet connection is necessary for the initial download of the required model weights. Although not mandatory, a CUDA-enabled GPU is recommended to enhance processing speed and support faster computations.

B.2 Installation Steps

To begin installation, users should first download or clone the Vision Morph AI project files into a working directory of their choice. After opening a terminal or command prompt within this directory, it is advisable to create and activate a Python virtual environment to isolate project dependencies. Following this, all required libraries and dependencies can be installed by running the command `pip install -r requirements.txt`. When running the program for the first time, it will automatically download the Intel DPT model weights from the Hugging Face repository and save them locally for subsequent use.

B.3 Running the Application

Starting the application involves navigating to the project folder and executing the command `python gui_app.py`. This launches the graphical user interface (GUI) window. Users can then load an image by clicking the "Browse" button, choosing from supported formats such as JPG, PNG, or BMP.

After selecting the desired conversion method—either "Heightmap" or "DPT Depth Estimation"—users have the option to adjust additional parameters including the Height Scale slider and smoothing or decimation settings as needed. Once satisfied with the settings, clicking the "Convert" button initiates the process, generating and displaying a preview of the 3D mesh. The finalized model can then be saved by clicking the "Export" button, which allows export in either OBJ or STL file formats.

B.4 Tips for Effective Use

For optimal performance, users should be aware that smaller images process more quickly, whereas larger images generate more detailed meshes but increase processing time and memory requirements. The Height Scale control influences the depth exaggeration of the generated model; it is advisable to begin with the default setting and adjust it gradually to achieve the desired effect. To manage file size and complexity, the decimation option can be used to simplify the mesh, facilitating easier handling in third-party software. To avoid errors related to missing libraries, users should ensure that all dependencies listed in the requirements.txt file have been fully installed before the first run.

B.5 Troubleshooting

If missing module errors occur, re-running the installation command `pip install -r requirements.txt` typically resolves the issue. Should the model download fail, users are advised to verify their internet connection and restart the program. In cases where an unsupported image format is identified, converting the image to JPG or PNG is recommended before loading it into the application. This user manual is designed to enable users to install, configure, and operate the Vision Morph AI system effectively without requiring direct reference to the source code.

References

Book References

1. Richard Szeliski, *Computer Vision: Algorithms and Applications*, 2nd Edition, Springer, 2022.
2. Ian Goodfellow, Yoshua Bengio, Aaron Courville, *Deep Learning*, MIT Press, 2016.
3. David Marr, *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*, MIT Press, 2010.
4. Hartley, R. and Zisserman, A., *Multiple View Geometry in Computer Vision*, 2nd Edition, Cambridge University Press, 2004.
5. Bishop, C. M., *Pattern Recognition and Machine Learning*, Springer, 2006.

Website References

1. Intel ISL – *MiDaS: Monocular Depth Estimation*
<https://github.com/isl-org/MiDaS>
2. Hugging Face Transformers Documentation – *Loading DPT Model*
<https://huggingface.co/docs/transformers/index>
3. Trimesh Documentation – *Python 3D Mesh Processing*
<https://trimsh.org/>
4. OpenCV Documentation – *Image Preprocessing and Handling*
<https://docs.opencv.org/>
5. Tkinter GUI Programming – *Python Official Docs*
<https://docs.python.org/3/library/tkinter.html>
6. PyVista Documentation – *3D Visualization Library*
<https://docs.pyvista.org/>